

Lab 4: VibeCoding - Space Invaders

Chat history with the AI assistant

USER

README.md
103 lines
MD
main.py
42 lines
PY
player.py
20 lines
PY
game_engine.py
105 lines
PY
enemy.py
55 lines
PY
bullet.py
20 lines
PY

I'm working on a Pygame Space Invaders project for a university lab (Python 3.10, pygame 2.6.1). Below are the README's expected behavior, my observations from running the game, and all of my code files. I will give you tasks one at a time. For each task, please:

- briefly explain the root cause or your approach
- give me the COMPLETE updated contents of only the files that changed
- do not change anything unrelated to the current task

EXPECTED BEHAVIOR (from the README):

- Smooth player movement using Left/Right or A/D, and shooting with Space
- Enemy grid marches side to side and drops down whenever it reaches a screen edge
- Enemies occasionally fire back at the player
- Score increases each time an enemy is destroyed
- Game ends when the player is hit by an enemy bullet or the enemy grid reaches the bottom of the screen

WHAT I OBSERVE WHEN I RUN IT:

- Enemies fire far too frequently (the screen fills with red bullets) and the game ends within about 3 seconds
- Some bullets register hits (score increased to 2), but the README says bullets sometimes pass through enemies, especially when firing rapidly
- When the game ends, it only prints "Game over! Final score: N" to the terminal and the window freezes on the last frame, then closes
- Some player bullets may not be moving or being removed correctly (not sure)

CODE FILES:

main.py:

[paste]

game/game_engine.py:

[paste]

game/player.py:

[paste]

game/enemy.py:

[paste]

game/bullet.py:

[paste]

TASK 0 (do this first): Enemies are firing too often and the game is unplayable. Review the enemy firing logic and fix it so enemies only "occasionally" fire back. Also tell me if you notice any other bugs in the code, but do not fix them yet.

ASSISTANT

Root cause: the firing check runs once per enemy, per frame. With 32 enemies, 60 FPS and a 1% chance each, that's about 0.32 new bullets per frame, or roughly 19 enemy bullets per second. That is why the screen fills with red and you die in about 3 seconds.

Fix: roll once per frame and, on success, let one randomly chosen alive enemy fire. At `enemy_fire_chance = 0.01` that averages about one enemy shot every 1.7 seconds, independent of how many enemies are alive. I kept the value at 0.01 so Task 3's difficulty levels can later tune it. I also left the existing collision loop and game-over code untouched for their own tasks.

Only `game/game_engine.py` changes (I used the files you uploaded, since the [paste] placeholders in your message were empty):

```
import pygame
import random
from .player import Player
from .enemy import EnemyGrid
from .bullet import Bullet

# Game Engine

WHITE = (255, 255, 255)
GREEN = (0, 200, 0)
RED = (220, 60, 60)

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.player = Player(width // 2 - 20, height - 50, 40, 20)
        self.enemy_grid = EnemyGrid(width)

        self.player_bullets = []
        self.enemy_bullets = []
        self._shoot_cooldown = 0
        # Chance PER FRAME that ONE randomly chosen enemy fires (~1 shot / 1.7s at 60 FPS)
        self.enemy_fire_chance = 0.01

        self.score = 0
        self.font = pygame.font.SysFont("Arial", 30)
        self.game_over = False

    def handle_event(self, event):
        if event.type == pygame.KEYDOWN and event.key == pygame.K_SPACE:
            if self._shoot_cooldown <= 0:
                bullet_x = self.player.center_x() - 2
                self.player_bullets.append(Bullet(bullet_x, self.player.y, direction=-1))
                self._shoot_cooldown = 15

    def handle_input(self):
        keys = pygame.key.get_pressed()
        if keys[pygame.K_LEFT] or keys[pygame.K_a]:
            self.player.move(-self.player.speed, self.width)
        if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
            self.player.move(self.player.speed, self.width)

    def update(self):
        if self.game_over:
            return
```

```

if self._shoot_cooldown > 0:
    self._shoot_cooldown -= 1

self.enemy_grid.move()

# Roll once per frame, then pick a single enemy to fire.
alive = self.enemy_grid.alive_enemies()
if alive and random.random() < self.enemy_fire_chance:
    shooter = random.choice(alive)
    bullet_x = shooter.x + shooter.width // 2
    self.enemy_bullets.append(Bullet(bullet_x, shooter.y + shooter.height, direction=1))

for bullet in self.player_bullets:
    bullet.move()
for bullet in self.enemy_bullets:
    bullet.move()

self.player_bullets = [b for b in self.player_bullets if not b.off_screen(self.height)]
self.enemy_bullets = [b for b in self.enemy_bullets if not b.off_screen(self.height)]

# NOTE: this removes a bullet from player_bullets while iterating
# directly over that same list. Python skips the element right
# after a removed one, so when two enemies are hit on the same
# frame the second collision can be missed - the bullet appears
# to pass straight through. See Task 1 in the README.
for bullet in self.player_bullets:
    for enemy in self.enemy_grid.alive_enemies():
        if bullet.rect().colliderect(enemy.rect()):
            enemy.alive = False
            self.player_bullets.remove(bullet)
            self.score += 1
            break

for bullet in self.enemy_bullets:
    if bullet.rect().colliderect(self.player.rect()):
        self.game_over = True
        break

if self.enemy_grid.reached_bottom(self.player.y):
    self.game_over = True

def render(self, screen):
    pygame.draw.rect(screen, GREEN, self.player.rect())

    for enemy in self.enemy_grid.alive_enemies():
        pygame.draw.rect(screen, WHITE, enemy.rect())

    for bullet in self.player_bullets:
        pygame.draw.rect(screen, WHITE, bullet.rect())
    for bullet in self.enemy_bullets:
        pygame.draw.rect(screen, RED, bullet.rect())

    score_text = self.font.render(f"Score: {self.score}", True, WHITE)
    screen.blit(score_text, (10, 10))

    if self.game_over and not getattr(self, "_game_over_logged", False):
        # NOTE: no proper game-over screen yet - see Task 2 in the README.
        print("Game over! Final score:", self.score)
        self._game_over_logged = True

```

Other bugs I noticed (not fixed yet)

Collision loop mutates the list it iterates (Task 1). `self.player_bullets.remove(bullet)` inside `for bullet in self.player_bullets` makes Python skip the next bullet that frame, so that bullet misses its hit check for one frame. This is the main cause of the "pass through" behavior. Bullet speed (8) and size (12) versus enemy height (30) rule out true tunneling.

Your question about bullets not moving or being removed correctly: player and enemy bullets do move and get culled off-screen properly. The only removal problem is the one in item 1.

The game isn't really frozen after game over. `update()` returns early, but `main.py` keeps running, and `handle_input()` and `handle_event()` still work, so you can move the ship and queue new player bullets that never move. Your "window closes" is probably you closing it, because nothing ends the loop.

Float positions in Rect. Enemy x becomes a float (speed 1.5), and `pygame.Rect` truncates it. Rendering and hitboxes are slightly off, but it's minor.

No win condition. If you destroy every enemy, nothing happens and the game just idles.

Missing files vs. README. `requirements.txt` wasn't in your upload (README references it), and `game/__init__.py` isn't listed. The import works anyway through Python 3's namespace packages.

Enemies fire from any row. A shooter can be hidden behind other enemies. Letting only the bottom enemy in each column fire would look more natural, but it's a design choice and I haven't changed it.

Ready for Task 1 whenever you are.

USER

Task 1: Refine collision detection. From the README: "Bullets sometimes pass straight through enemies without registering a hit, especially when firing rapidly or when two enemies are hit close together. Investigate and enhance collision accuracy."

You identified that removing bullets from `self.player_bullets` while iterating over it skips the next bullet. Please fix that properly (iterate over a copy, or build a list of items to remove and remove them after the loop). Apply the same fix to enemy bullets and to enemy removal. Also make sure one bullet can only destroy one enemy, and that two bullets hitting two adjacent enemies in the same frame both register. Use `pygame.Rect.colliderect` for hit checks. Give me the complete updated files that change, and don't touch the game-over logic yet.

ASSISTANT

Root cause: `self.player_bullets.remove(bullet)` ran inside for bullet in `self.player_bullets`. Removing an item shifts the rest of the list left, so the iterator skips the next bullet for that frame. When two bullets hit on the same frame, the second one can be missed, which looks like it passed through.

Fix: The collision pass now has three steps:

Detect. Loop over the bullets (nothing is mutated here) and record which enemies and bullets were hit.

One bullet, one enemy. Each bullet stops at the first enemy it overlaps (break). An enemy already claimed by an earlier bullet this frame is skipped, so one enemy can't be scored twice and a second bullet can still hit its neighbor.

Apply. After the loop, mark the enemies dead, add the score, and rebuild `player_bullets` without the spent bullets.

Enemy removal and enemy bullets: Neither mutates a list while iterating. Enemies are removed by setting `alive = False`, and `alive_enemies()` returns a fresh list each call. The enemy-bullet loop never removes anything, it only checks for a player hit. So there was nothing to repair there. The new enemy removal is just batched after detection so it can't affect the checks in the same frame. I left the enemy-bullet and game-over code untouched, as requested.

Only `game/game_engine.py` changes (this keeps the Task 0 firing fix):

```
import pygame
import random
from .player import Player
from .enemy import EnemyGrid
from .bullet import Bullet

# Game Engine

WHITE = (255, 255, 255)
GREEN = (0, 200, 0)
RED = (220, 60, 60)

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.player = Player(width // 2 - 20, height - 50, 40, 20)
        self.enemy_grid = EnemyGrid(width)

        self.player_bullets = []
        self.enemy_bullets = []
        self._shoot_cooldown = 0
        # Chance PER FRAME that ONE randomly chosen enemy fires (~1 shot / 1.7s at 60 FPS)
        self.enemy_fire_chance = 0.01

        self.score = 0
        self.font = pygame.font.SysFont("Arial", 30)
        self.game_over = False

    def handle_event(self, event):
        if event.type == pygame.KEYDOWN and event.key == pygame.K_SPACE:
            if self._shoot_cooldown <= 0:
                bullet_x = self.player.center_x() - 2
```

```

        self.player_bullets.append(Bullet(bullet_x, self.player.y, direction=-1))
        self._shoot_cooldown = 15

def handle_input(self):
    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT] or keys[pygame.K_a]:
        self.player.move(-self.player.speed, self.width)
    if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
        self.player.move(self.player.speed, self.width)

def _resolve_player_bullet_hits(self):
    """Check every player bullet against every alive enemy.

    Nothing is removed while looping. Hits are collected first and
    applied afterwards, so every bullet gets checked every frame.
    Each bullet can destroy at most one enemy, and each enemy can be
    destroyed (and scored) at most once per frame.
    """
    enemies = self.enemy_grid.alive_enemies()
    hit_enemies = set()
    spent_bullets = set()

    for bullet in self.player_bullets:
        bullet_rect = bullet.rect()
        for enemy in enemies:
            if enemy in hit_enemies:
                continue # already claimed by another bullet this frame
            if bullet_rect.colliderect(enemy.rect()):
                hit_enemies.add(enemy)
                spent_bullets.add(bullet)
                break # one bullet destroys only one enemy

    # Apply results after all detection is done
    for enemy in hit_enemies:
        enemy.alive = False
    self.score += len(hit_enemies)
    self.player_bullets = [b for b in self.player_bullets if b not in spent_bullets]

def update(self):
    if self.game_over:
        return

    if self._shoot_cooldown > 0:
        self._shoot_cooldown -= 1

    self.enemy_grid.move()

    # Roll once per frame, then pick a single enemy to fire.
    alive = self.enemy_grid.alive_enemies()
    if alive and random.random() < self.enemy_fire_chance:
        shooter = random.choice(alive)
        bullet_x = shooter.x + shooter.width // 2
        self.enemy_bullets.append(Bullet(bullet_x, shooter.y + shooter.height, direction=1))

    for bullet in self.player_bullets:
        bullet.move()
    for bullet in self.enemy_bullets:
        bullet.move()

    self.player_bullets = [b for b in self.player_bullets if not b.off_screen(self.height)]
    self.enemy_bullets = [b for b in self.enemy_bullets if not b.off_screen(self.height)]

    self._resolve_player_bullet_hits()

    for bullet in self.enemy_bullets:
        if bullet.rect().colliderect(self.player.rect()):
            self.game_over = True
            break

    if self.enemy_grid.reached_bottom(self.player.y):
        self.game_over = True

def render(self, screen):
    pygame.draw.rect(screen, GREEN, self.player.rect())

    for enemy in self.enemy_grid.alive_enemies():
        pygame.draw.rect(screen, WHITE, enemy.rect())

    for bullet in self.player_bullets:
        pygame.draw.rect(screen, WHITE, bullet.rect())
    for bullet in self.enemy_bullets:
        pygame.draw.rect(screen, RED, bullet.rect())

```

```

score_text = self.font.render(f"Score: {self.score}", True, WHITE)
screen.blit(score_text, (10, 10))

if self.game_over and not getattr(self, "_game_over_logged", False):
    # NOTE: no proper game-over screen yet - see Task 2 in the README.
    print("Game over! Final score:", self.score)
    self._game_over_logged = True

```

Testing this for your "after" video:

Fire at the bottom row. Each bullet should kill exactly one enemy, and the score should go up by exactly 1 per kill.

Rapid fire is capped by the 15-frame cooldown (about 4 shots per second). Two bullets can still be in flight a few rows apart, so you can fire two shots in a row at neighboring columns and both should register.

Enemy, Bullet, and Player have no `__eq__`/`__hash__` overrides, so the `set()` usage relies on object identity. That is what we want here.

One limitation: there is still no swept collision test. At bullet speed 8 and enemy height 30, tunneling can't happen, so I left it out. If you raise bullet speed past about 30 px/frame in Task 3, that would need revisiting.

Ready for Task 2 when you are.

USER

Task 2: Implement a Game Over condition. From the README: "Add a screen that displays the final score once the player is hit or the enemies reach the bottom of the screen, then gracefully waits for input instead of just printing to the console."

Requirements:

- Draw the Game Over screen inside the pygame window: "GAME OVER" title, the final score, and a prompt such as "Press any key to continue"
- When game over happens, stop updating the game world, and don't let the player keep moving or queueing bullets
- Wait for a key press (or window close) without freezing the window or crashing, and exit cleanly
- Remove the console print of the final score
- Structure the code so Task 3 (replay with Easy/Medium/Hard difficulty) can be added next without a rewrite, for example by keeping a reset/new-game method in GameEngine

Give me the complete updated files that change (game_engine.py and main.py if needed). Don't change collision or enemy-firing logic.

ASSISTANT

Approach: I added a game-over state to GameEngine that freezes the world, ignores gameplay input, draws an overlay inside the window, and waits for a key press or window close. The console print is gone. Two details go beyond your list:

Input delay: The prompt and key handling only become active about 0.5 seconds after the game ends. Otherwise a player who is mashing Space or an arrow key when they die would dismiss the screen instantly without seeing their score.

Task 3 hook: `__init__` now calls a new `reset()` method that rebuilds all world state (player, enemies, bullets, score, cooldown). For Task 3, the key handler in the game-over state is the only place that changes: instead of setting `should_quit`, it will show the difficulty menu and call `reset()` with the chosen settings.

Collision and enemy-firing logic are untouched. `main.py` changes only to stop the loop when the engine asks to quit.

game/game_engine.py

```

import pygame
import random
from .player import Player
from .enemy import EnemyGrid
from .bullet import Bullet

# Game Engine

WHITE = (255, 255, 255)
GREEN = (0, 200, 0)
RED = (220, 60, 60)

GAME_OVER_INPUT_DELAY = 30 # frames (~0.5s at 60 FPS) before a key press is accepted

```

```

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.font = pygame.font.SysFont("Arial", 30)
        self.title_font = pygame.font.SysFont("Arial", 64, bold=True)
        self.prompt_font = pygame.font.SysFont("Arial", 24)

        self.should_quit = False
        self.reset()

    def reset(self):
        """Start a fresh game. Rebuilds all world state."""
        self.player = Player(self.width // 2 - 20, self.height - 50, 40, 20)
        self.enemy_grid = EnemyGrid(self.width)

        self.player_bullets = []
        self.enemy_bullets = []
        self._shoot_cooldown = 0
        # Chance PER FRAME that ONE randomly chosen enemy fires (~1 shot / 1.7s at 60 FPS)
        self.enemy_fire_chance = 0.01

        self.score = 0
        self.game_over = False
        self._game_over_timer = 0

    def _trigger_game_over(self):
        if not self.game_over:
            self.game_over = True
            self._game_over_timer = GAME_OVER_INPUT_DELAY

    def _game_over_ready(self):
        return self.game_over and self._game_over_timer <= 0

    def handle_event(self, event):
        if self.game_over:
            # Ignore gameplay input; wait for any key once the delay has passed.
            if event.type == pygame.KEYDOWN and self._game_over_ready():
                self.should_quit = True # Task 3 will show the replay menu here instead
                return

            if event.type == pygame.KEYDOWN and event.key == pygame.K_SPACE:
                if self._shoot_cooldown <= 0:
                    bullet_x = self.player.center_x() - 2
                    self.player_bullets.append(Bullet(bullet_x, self.player.y, direction=-1))
                    self._shoot_cooldown = 15

    def handle_input(self):
        if self.game_over:
            return

        keys = pygame.key.get_pressed()
        if keys[pygame.K_LEFT] or keys[pygame.K_a]:
            self.player.move(-self.player.speed, self.width)
        if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
            self.player.move(self.player.speed, self.width)

    def _resolve_player_bullet_hits(self):
        """Check every player bullet against every alive enemy.

        Nothing is removed while looping. Hits are collected first and
        applied afterwards, so every bullet gets checked every frame.
        Each bullet can destroy at most one enemy, and each enemy can be
        destroyed (and scored) at most once per frame.
        """
        enemies = self.enemy_grid.alive_enemies()
        hit_enemies = set()
        spent_bullets = set()

        for bullet in self.player_bullets:
            bullet_rect = bullet.rect()
            for enemy in enemies:
                if enemy in hit_enemies:
                    continue # already claimed by another bullet this frame
                if bullet_rect.colliderect(enemy.rect()):
                    hit_enemies.add(enemy)
                    spent_bullets.add(bullet)
                    break # one bullet destroys only one enemy

        # Apply results after all detection is done
        for enemy in hit_enemies:

```

```

        enemy.alive = False
    self.score += len(hit_enemies)
    self.player_bullets = [b for b in self.player_bullets if b not in spent_bullets]

def update(self):
    if self.game_over:
        # World is frozen; only count down the input delay.
        if self._game_over_timer > 0:
            self._game_over_timer -= 1
        return

    if self._shoot_cooldown > 0:
        self._shoot_cooldown -= 1

    self.enemy_grid.move()

    # Roll once per frame, then pick a single enemy to fire.
    alive = self.enemy_grid.alive_enemies()
    if alive and random.random() < self.enemy_fire_chance:
        shooter = random.choice(alive)
        bullet_x = shooter.x + shooter.width // 2
        self.enemy_bullets.append(Bullet(bullet_x, shooter.y + shooter.height, direction=1))

    for bullet in self.player_bullets:
        bullet.move()
    for bullet in self.enemy_bullets:
        bullet.move()

    self.player_bullets = [b for b in self.player_bullets if not b.off_screen(self.height)]
    self.enemy_bullets = [b for b in self.enemy_bullets if not b.off_screen(self.height)]

    self._resolve_player_bullet_hits()

    for bullet in self.enemy_bullets:
        if bullet.rect().colliderect(self.player.rect()):
            self._trigger_game_over()
            break

    if self.enemy_grid.reached_bottom(self.player.y):
        self._trigger_game_over()

def _render_game_over(self, screen):
    # Dim the frozen game behind the text
    overlay = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
    overlay.fill((0, 0, 0, 160))
    screen.blit(overlay, (0, 0))

    cx, cy = self.width // 2, self.height // 2

    title = self.title_font.render("GAME OVER", True, RED)
    screen.blit(title, title.get_rect(center=(cx, cy - 60)))

    score = self.font.render(f"Final Score: {self.score}", True, WHITE)
    screen.blit(score, score.get_rect(center=(cx, cy + 10)))

    # Prompt appears once input is being accepted
    if self._game_over_ready():
        prompt = self.prompt_font.render("Press any key to continue", True, WHITE)
        screen.blit(prompt, prompt.get_rect(center=(cx, cy + 70)))

def render(self, screen):
    pygame.draw.rect(screen, GREEN, self.player.rect())

    for enemy in self.enemy_grid.alive_enemies():
        pygame.draw.rect(screen, WHITE, enemy.rect())

    for bullet in self.player_bullets:
        pygame.draw.rect(screen, WHITE, bullet.rect())
    for bullet in self.enemy_bullets:
        pygame.draw.rect(screen, RED, bullet.rect())

    score_text = self.font.render(f"Score: {self.score}", True, WHITE)
    screen.blit(score_text, (10, 10))

    if self.game_over:
        self._render_game_over(screen)

```

main.py

```

import pygame
from game.game_engine import GameEngine

```

```

# Initialize pygame/Start application
pygame.init()

# Screen dimensions
WIDTH, HEIGHT = 600, 700
SCREEN = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Space Invaders - Pygame Version")

# Colors
BLACK = (0, 0, 0)

# Clock
clock = pygame.time.Clock()
FPS = 60

# Game loop
engine = GameEngine(WIDTH, HEIGHT)

def main():
    running = True
    while running:
        SCREEN.fill(BLACK)
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            engine.handle_event(event)

        engine.handle_input()
        engine.update()
        engine.render(SCREEN)

        if engine.should_quit:
            running = False

        pygame.display.flip()
        clock.tick(FPS)

    pygame.quit()

if __name__ == "__main__":
    main()

```

What to check for your "after" video:

Get hit by a red bullet (or let the grid reach the bottom). The world should freeze under a dimmed overlay showing "GAME OVER" and "Final Score: N".

Moving the ship or pressing Space on that screen should do nothing.

After about half a second, "Press any key to continue" appears. Pressing any key, or closing the window, exits cleanly.

The terminal should print nothing.

Quirk to know about: `should_quit` currently just ends the program, since there is no replay yet. Task 3 will replace that one line.

Ready for Task 3 when you are.

USER

Task 3: Add a replay option. From the README: "After Game Over, allow the user to play again by choosing a difficulty (Easy, Medium, or Hard enemy speed/fire rate), or exit."

Requirements:

- After the game over delay, replace the "Press any key to continue" step with a menu drawn inside the pygame window: choose Easy, Medium, or Hard, or Exit
- Allow selection with Up/Down (or W/S) plus Enter, and also number keys 1/2/3 for difficulty and Esc (or 4) for exit. Show the keys on screen
- Difficulty must change both enemy movement speed and enemy fire rate. Please show me the exact values you chose for Easy, Medium and Hard, and keep Medium close to the current default. If enemy speed is set in `enemy.py`, update that file too
- On replay, call `reset()` so the score, enemies, bullets, player and cooldown are all fully reset, then apply the chosen difficulty
- Exit and window close must quit cleanly
- Don't change collision logic

Give me the complete updated files that change.

ASSISTANT

Approach: After the 0.5 s delay, the game-over screen now shows a menu instead of the "press any key" prompt. Choosing a difficulty calls `reset(difficulty)`, which rebuilds the player, enemies, bullets, score and cooldown, then applies that difficulty's settings.

No change to `enemy.py` or `main.py`. `EnemyGrid` already takes a speed argument, so `reset()` passes it in. `main.py` already quits on `should_quit` and on window close. Collision logic is untouched.

Difficulty values:

	Enemy speed (px/frame)	Fire chance per frame	Approx. enemy shots
Easy	1.0	0.006	1 every ~2.8 s
Medium	1.5 (current default)	0.01 (current default)	1 every ~1.7 s
Hard	2.5	0.02	1 every ~0.8 s

The firing roll is the Task 0 version: one roll per frame, then one random enemy fires. The menu opens with your last-played difficulty highlighted. The first game starts on Medium.

`game/game_engine.py`

```
import pygame
import random
from .player import Player
from .enemy import EnemyGrid
from .bullet import Bullet

# Game Engine

WHITE = (255, 255, 255)
GREEN = (0, 200, 0)
RED = (220, 60, 60)
YELLOW = (255, 220, 60)
GREY = (170, 170, 170)

GAME_OVER_INPUT_DELAY = 30 # frames (~0.5s at 60 FPS) before menu input is accepted

# enemy_speed: pixels per frame for the enemy grid
# fire_chance: chance PER FRAME that ONE randomly chosen enemy fires
DIFFICULTIES = {
    "Easy": {"enemy_speed": 1.0, "fire_chance": 0.006},
    "Medium": {"enemy_speed": 1.5, "fire_chance": 0.01},
    "Hard": {"enemy_speed": 2.5, "fire_chance": 0.02},
}

# Menu entries in display order: (label, key hint)
MENU_OPTIONS = [
    ("Easy", "1"),
    ("Medium", "2"),
    ("Hard", "3"),
    ("Exit", "4 / Esc"),
]

EXIT_INDEX = 3

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.font = pygame.font.SysFont("Arial", 30)
        self.title_font = pygame.font.SysFont("Arial", 64, bold=True)
        self.prompt_font = pygame.font.SysFont("Arial", 22)

        self.should_quit = False
        self.difficulty = "Medium"
        self.reset()

    def reset(self, difficulty=None):
        """Start a fresh game. Rebuilds all world state.

        If a difficulty name is given it becomes the current difficulty;
        otherwise the current one is reused.
        """
        if difficulty is not None:
            self.difficulty = difficulty
```

```

settings = DIFFICULTIES[self.difficulty]

self.player = Player(self.width // 2 - 20, self.height - 50, 40, 20)
self.enemy_grid = EnemyGrid(self.width, speed=settings["enemy_speed"])

self.player_bullets = []
self.enemy_bullets = []
self._shoot_cooldown = 0
# Chance PER FRAME that ONE randomly chosen enemy fires
self.enemy_fire_chance = settings["fire_chance"]

self.score = 0
self.game_over = False
self._game_over_timer = 0
self._menu_index = 0

def _trigger_game_over(self):
    if not self.game_over:
        self.game_over = True
        self._game_over_timer = GAME_OVER_INPUT_DELAY
        # Start the menu highlight on the difficulty just played
        self._menu_index = list(DIFFICULTIES).index(self.difficulty)

def _game_over_ready(self):
    return self.game_over and self._game_over_timer <= 0

def _activate_menu_option(self, index):
    if index == EXIT_INDEX:
        self.should_quit = True
    else:
        self.reset(MENU_OPTIONS[index][0])

def _handle_menu_key(self, key):
    if key in (pygame.K_UP, pygame.K_w):
        self._menu_index = (self._menu_index - 1) % len(MENU_OPTIONS)
    elif key in (pygame.K_DOWN, pygame.K_s):
        self._menu_index = (self._menu_index + 1) % len(MENU_OPTIONS)
    elif key in (pygame.K_RETURN, pygame.K_KP_ENTER):
        self._activate_menu_option(self._menu_index)
    elif key in (pygame.K_1, pygame.K_KP1):
        self._activate_menu_option(0)
    elif key in (pygame.K_2, pygame.K_KP2):
        self._activate_menu_option(1)
    elif key in (pygame.K_3, pygame.K_KP3):
        self._activate_menu_option(2)
    elif key in (pygame.K_4, pygame.K_KP4, pygame.K_ESCAPE):
        self._activate_menu_option(EXIT_INDEX)

def handle_event(self, event):
    if self.game_over:
        # Ignore gameplay input; menu keys only work once the delay has passed.
        if event.type == pygame.KEYDOWN and self._game_over_ready():
            self._handle_menu_key(event.key)
        return

    if event.type == pygame.KEYDOWN and event.key == pygame.K_SPACE:
        if self._shoot_cooldown <= 0:
            bullet_x = self.player.center_x() - 2
            self.player_bullets.append(Bullet(bullet_x, self.player.y, direction=-1))
            self._shoot_cooldown = 15

def handle_input(self):
    if self.game_over:
        return

    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT] or keys[pygame.K_a]:
        self.player.move(-self.player.speed, self.width)
    if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
        self.player.move(self.player.speed, self.width)

def _resolve_player_bullet_hits(self):
    """Check every player bullet against every alive enemy.

    Nothing is removed while looping. Hits are collected first and
    applied afterwards, so every bullet gets checked every frame.
    Each bullet can destroy at most one enemy, and each enemy can be
    destroyed (and scored) at most once per frame.
    """
    enemies = self.enemy_grid.alive_enemies()
    hit_enemies = set()
    spent_bullets = set()

```

```

for bullet in self.player_bullets:
    bullet_rect = bullet.rect()
    for enemy in enemies:
        if enemy in hit_enemies:
            continue # already claimed by another bullet this frame
        if bullet_rect.colliderect(enemy.rect()):
            hit_enemies.add(enemy)
            spent_bullets.add(bullet)
            break # one bullet destroys only one enemy

# Apply results after all detection is done
for enemy in hit_enemies:
    enemy.alive = False
self.score += len(hit_enemies)
self.player_bullets = [b for b in self.player_bullets if b not in spent_bullets]

def update(self):
    if self.game_over:
        # World is frozen; only count down the input delay.
        if self._game_over_timer > 0:
            self._game_over_timer -= 1
        return

    if self._shoot_cooldown > 0:
        self._shoot_cooldown -= 1

    self.enemy_grid.move()

    # Roll once per frame, then pick a single enemy to fire.
    alive = self.enemy_grid.alive_enemies()
    if alive and random.random() < self.enemy_fire_chance:
        shooter = random.choice(alive)
        bullet_x = shooter.x + shooter.width // 2
        self.enemy_bullets.append(Bullet(bullet_x, shooter.y + shooter.height, direction=1))

    for bullet in self.player_bullets:
        bullet.move()
    for bullet in self.enemy_bullets:
        bullet.move()

    self.player_bullets = [b for b in self.player_bullets if not b.off_screen(self.height)]
    self.enemy_bullets = [b for b in self.enemy_bullets if not b.off_screen(self.height)]

    self._resolve_player_bullet_hits()

    for bullet in self.enemy_bullets:
        if bullet.rect().colliderect(self.player.rect()):
            self._trigger_game_over()
            break

    if self.enemy_grid.reached_bottom(self.player.y):
        self._trigger_game_over()

def _render_game_over(self, screen):
    # Dim the frozen game behind the text
    overlay = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
    overlay.fill((0, 0, 0, 160))
    screen.blit(overlay, (0, 0))

    cx, cy = self.width // 2, self.height // 2

    title = self.title_font.render("GAME OVER", True, RED)
    screen.blit(title, title.get_rect(center=(cx, cy - 160)))

    score = self.font.render(f"Final Score: {self.score}", True, WHITE)
    screen.blit(score, score.get_rect(center=(cx, cy - 100)))

    # Menu appears once input is being accepted
    if not self._game_over_ready():
        return

    header = self.prompt_font.render("Play again? Choose a difficulty:", True, WHITE)
    screen.blit(header, header.get_rect(center=(cx, cy - 45)))

    for i, (label, hint) in enumerate(MENU_OPTIONS):
        selected = (i == self._menu_index)
        color = YELLOW if selected else WHITE
        marker = "> " if selected else " "
        y = cy + i * 45

        text = self.font.render(f"{marker}{label}", True, color)
        screen.blit(text, text.get_rect(midleft=(cx - 130, y)))

```

```

        hint_text = self.prompt_font.render(f"[{hint}]", True, GREY)
        screen.blit(hint_text, hint_text.get_rect(midright=(cx + 140, y)))

    help_text = self.prompt_font.render("Up/Down or W/S to move, Enter to select", True, GREY)
    screen.blit(help_text, help_text.get_rect(center=(cx, cy + 4 * 45 + 20)))

def render(self, screen):
    pygame.draw.rect(screen, GREEN, self.player.rect())

    for enemy in self.enemy_grid.alive_enemies():
        pygame.draw.rect(screen, WHITE, enemy.rect())

    for bullet in self.player_bullets:
        pygame.draw.rect(screen, WHITE, bullet.rect())
    for bullet in self.enemy_bullets:
        pygame.draw.rect(screen, RED, bullet.rect())

    score_text = self.font.render(f"Score: {self.score}", True, WHITE)
    screen.blit(score_text, (10, 10))

    if self.game_over:
        self._render_game_over(screen)

```

What to check for your "after" video:

After dying, "GAME OVER" and the final score appear immediately. About half a second later the menu appears with your last difficulty highlighted.

Up/Down or W/S move the highlight and wrap around. Enter selects it. 1/2/3 start Easy/Medium/Hard directly. Esc or 4 exits. The key hints are shown next to each option.

After choosing, the score is 0, the enemy grid is back at the top, there are no leftover bullets, and the ship is centered.

Hard should feel clearly faster, with enemies shooting more often. Easy should feel noticeably gentler.

Closing the window at any point quits cleanly.

Things to know:

The enemy drop amount (20 px) is the same on all difficulties, so only speed and fire rate differ.

On Hard the grid moves 2.5 px/frame, and with the Task 0 firing logic the fire rate is independent of how many enemies are left. Late-game Hard can feel dense, since a few remaining enemies still fire about once every 0.8 s. If that is too harsh, lower Hard's fire_chance to about 0.015.

Ready for Task 4 when you are.

USER

Task 4: Add basic sound effects. From the README: "Add basic sound effects for firing, an enemy being destroyed, and the game-over moment."

Requirements:

- Sounds for: player firing, enemy destroyed, and the game-over moment (play the game-over sound once when the game ends, not every frame)
- I don't have any audio files, so generate the sounds in code. Prefer a method that doesn't need new dependencies, for example building the waveform with the standard library (array / math) and pygame.mixer.Sound(buffer=...). If you need numpy, tell me and also update requirements.txt
- Handle audio failure gracefully: if pygame.mixer can't initialize (no sound device), the game must still run without crashing
- Put the sound code in a small separate file, such as game/sounds.py, and call it from game_engine.py so the engine stays readable
- Don't change collision, firing, difficulty or menu logic, and don't play any sound when a replay starts

Give me the complete updated files that change, and tell me exactly which new files to create.

ASSISTANT

Approach: Sounds are built at startup with only math, array and random, so there are no new dependencies and requirements.txt stays unchanged. A new SoundManager class in game/sounds.py owns all the audio, and the engine only calls self.sounds.play("fire") and similar.

Files

Create: game/sounds.py (new, contents below)

Replace: game/game_engine.py (complete file below)

Unchanged: main.py, enemy.py, player.py, bullet.py, requirements.txt

Design notes:

Matching the mixer format: `pygame.mixer.Sound(buffer=...)` expects raw samples in the mixer's own format. The manager reads the actual frequency and channel count from `pygame.mixer.get_init()` and writes 16-bit samples, duplicated per channel if the mixer is stereo. If the mixer is in some other sample format, sound is disabled instead of playing noise.

Graceful failure: `pygame.init()` silently skips the mixer if there is no audio device. `SoundManager` then tries `pygame.mixer.init()` once and, if that also fails, sets `enabled = False`. `play()` becomes a no-op, and it is also wrapped in `try/except`, so the game never crashes over audio.

Own random generator: The explosion noise uses a private `random.Random`, so generating sounds doesn't touch the global random state that drives enemy firing.

Play-once rules: The game-over sound sits inside the `if not self.game_over` block of `_trigger_game_over()`, so it plays exactly once per game over. Nothing plays in `reset()`, so replays are silent. Fire plays only when a bullet is actually created, not when Space is blocked by the cooldown. Enemy-destroyed plays once per frame in which at least one enemy dies.

Logic untouched: Collision, firing, difficulty and menu logic are unchanged. The only edits in `game_engine.py` are the import, the `SoundManager` creation, and three `play()` calls.

game/sounds.py (new file)

```
import math
import random
from array import array

import pygame

MASTER_VOLUME = 0.35 # 0.0 - 1.0, keeps the generated sounds from being harsh

class SoundManager:
    """Builds simple sound effects in code and plays them.

    If audio can't be initialised, `enabled` is False and play() does nothing,
    so the rest of the game never has to care.
    """

    def __init__(self):
        self.enabled = False
        self._sounds = {}
        self._rng = random.Random(1234) # private RNG; doesn't touch the game's random state

        try:
            if pygame.mixer.get_init() is None:
                pygame.mixer.init()
            init = pygame.mixer.get_init()
            if init is None:
                return
            self._rate, fmt, self._channels = init
            if fmt != -16: # we only generate signed 16-bit samples
                return

            self._sounds = {
                "fire": self._make(self._fire_samples()),
                "explosion": self._make(self._explosion_samples()),
                "game_over": self._make(self._game_over_samples()),
            }
            self.enabled = True
        except Exception:
            self.enabled = False
            self._sounds = {}

    # ----- public API -----

    def play(self, name):
        if not self.enabled:
            return
        try:
            self._sounds[name].play()
```

```

    except Exception:
        pass # never let audio problems crash the game

# ----- sample generation -----

def _make(self, samples):
    """Turn a list of floats in [-1, 1] into a pygame Sound."""
    data = array("h")
    for s in samples:
        value = int(max(-1.0, min(1.0, s)) * MASTER_VOLUME * 32767)
        for _ in range(self._channels): # duplicate for stereo mixers
            data.append(value)
    return pygame.mixer.Sound(buffer=data.tobytes())

def _envelope(self, i, n, attack=0.005, release_power=1.5):
    """Short fade-in (avoids clicks) and a decaying tail."""
    t = i / self._rate
    attack_gain = min(1.0, t / attack) if attack > 0 else 1.0
    tail_gain = (1.0 - i / n) ** release_power
    return attack_gain * tail_gain

def _sweep(self, f_start, f_end, duration, square_mix=0.0):
    """Tone gliding from f_start to f_end Hz; square_mix adds a buzzy edge."""
    n = int(self._rate * duration)
    phase = 0.0
    out = []
    for i in range(n):
        freq = f_start + (f_end - f_start) * (i / n)
        phase += 2 * math.pi * freq / self._rate
        sine = math.sin(phase)
        square = 1.0 if sine >= 0 else -1.0
        wave = sine * (1 - square_mix) + square * square_mix
        out.append(wave * self._envelope(i, n))
    return out

def _fire_samples(self):
    # Quick descending "pew"
    return self._sweep(900, 300, 0.12, square_mix=0.4)

def _explosion_samples(self):
    # Noise burst plus a low falling thump
    duration = 0.25
    n = int(self._rate * duration)
    thump = self._sweep(160, 50, duration)
    out = []
    for i in range(n):
        noise = self._rng.uniform(-1.0, 1.0) * self._envelope(i, n, release_power=2.0)
        out.append(noise * 0.7 + thump[i] * 0.6)
    return out

def _game_over_samples(self):
    # Four descending notes, slightly buzzy
    out = []
    for freq in (440, 370, 311, 220):
        out.extend(self._sweep(freq, freq * 0.97, 0.22, square_mix=0.3))
    return out

```

game/game_engine.py

```

import pygame
import random
from .player import Player
from .enemy import EnemyGrid
from .bullet import Bullet
from .sounds import SoundManager

# Game Engine

WHITE = (255, 255, 255)
GREEN = (0, 200, 0)
RED = (220, 60, 60)
YELLOW = (255, 220, 60)
GREY = (170, 170, 170)

GAME_OVER_INPUT_DELAY = 30 # frames (~0.5s at 60 FPS) before menu input is accepted

# enemy_speed: pixels per frame for the enemy grid
# fire_chance: chance PER FRAME that ONE randomly chosen enemy fires
DIFFICULTIES = {
    "Easy": {"enemy_speed": 1.0, "fire_chance": 0.006},
    "Medium": {"enemy_speed": 1.5, "fire_chance": 0.01},
    "Hard": {"enemy_speed": 2.5, "fire_chance": 0.02},
}

```

```

}

# Menu entries in display order: (label, key hint)
MENU_OPTIONS = [
    ("Easy", "1"),
    ("Medium", "2"),
    ("Hard", "3"),
    ("Exit", "4 / Esc"),
]
EXIT_INDEX = 3

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.font = pygame.font.SysFont("Arial", 30)
        self.title_font = pygame.font.SysFont("Arial", 64, bold=True)
        self.prompt_font = pygame.font.SysFont("Arial", 22)

        self.sounds = SoundManager()

        self.should_quit = False
        self.difficulty = "Medium"
        self.reset()

    def reset(self, difficulty=None):
        """Start a fresh game. Rebuilds all world state.

        If a difficulty name is given it becomes the current difficulty;
        otherwise the current one is reused.
        """
        if difficulty is not None:
            self.difficulty = difficulty
        settings = DIFFICULTIES[self.difficulty]

        self.player = Player(self.width // 2 - 20, self.height - 50, 40, 20)
        self.enemy_grid = EnemyGrid(self.width, speed=settings["enemy_speed"])

        self.player_bullets = []
        self.enemy_bullets = []
        self._shoot_cooldown = 0
        # Chance PER FRAME that ONE randomly chosen enemy fires
        self.enemy_fire_chance = settings["fire_chance"]

        self.score = 0
        self.game_over = False
        self._game_over_timer = 0
        self._menu_index = 0

    def _trigger_game_over(self):
        if not self.game_over:
            self.game_over = True
            self._game_over_timer = GAME_OVER_INPUT_DELAY
            # Start the menu highlight on the difficulty just played
            self._menu_index = list(DIFFICULTIES).index(self.difficulty)
            self.sounds.play("game_over") # inside the guard, so it plays once

    def _game_over_ready(self):
        return self.game_over and self._game_over_timer <= 0

    def _activate_menu_option(self, index):
        if index == EXIT_INDEX:
            self.should_quit = True
        else:
            self.reset(MENU_OPTIONS[index][0])

    def _handle_menu_key(self, key):
        if key in (pygame.K_UP, pygame.K_w):
            self._menu_index = (self._menu_index - 1) % len(MENU_OPTIONS)
        elif key in (pygame.K_DOWN, pygame.K_s):
            self._menu_index = (self._menu_index + 1) % len(MENU_OPTIONS)
        elif key in (pygame.K_RETURN, pygame.K_KP_ENTER):
            self._activate_menu_option(self._menu_index)
        elif key in (pygame.K_1, pygame.K_KP1):
            self._activate_menu_option(0)
        elif key in (pygame.K_2, pygame.K_KP2):
            self._activate_menu_option(1)
        elif key in (pygame.K_3, pygame.K_KP3):
            self._activate_menu_option(2)
        elif key in (pygame.K_4, pygame.K_KP4, pygame.K_ESCAPE):
            self._activate_menu_option(EXIT_INDEX)

```

```

def handle_event(self, event):
    if self.game_over:
        # Ignore gameplay input; menu keys only work once the delay has passed.
        if event.type == pygame.KEYDOWN and self._game_over_ready():
            self._handle_menu_key(event.key)
        return

    if event.type == pygame.KEYDOWN and event.key == pygame.K_SPACE:
        if self._shoot_cooldown <= 0:
            bullet_x = self.player.center_x() - 2
            self.player_bullets.append(Bullet(bullet_x, self.player.y, direction=-1))
            self._shoot_cooldown = 15
            self.sounds.play("fire")

def handle_input(self):
    if self.game_over:
        return

    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT] or keys[pygame.K_a]:
        self.player.move(-self.player.speed, self.width)
    if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
        self.player.move(self.player.speed, self.width)

def _resolve_player_bullet_hits(self):
    """Check every player bullet against every alive enemy.

    Nothing is removed while looping. Hits are collected first and
    applied afterwards, so every bullet gets checked every frame.
    Each bullet can destroy at most one enemy, and each enemy can be
    destroyed (and scored) at most once per frame.
    """
    enemies = self.enemy_grid.alive_enemies()
    hit_enemies = set()
    spent_bullets = set()

    for bullet in self.player_bullets:
        bullet_rect = bullet.rect()
        for enemy in enemies:
            if enemy in hit_enemies:
                continue # already claimed by another bullet this frame
            if bullet_rect.colliderect(enemy.rect()):
                hit_enemies.add(enemy)
                spent_bullets.add(bullet)
                break # one bullet destroys only one enemy

    # Apply results after all detection is done
    for enemy in hit_enemies:
        enemy.alive = False
    self.score += len(hit_enemies)
    self.player_bullets = [b for b in self.player_bullets if b not in spent_bullets]

    if hit_enemies:
        self.sounds.play("explosion") # once per frame, even for a double hit

def update(self):
    if self.game_over:
        # World is frozen; only count down the input delay.
        if self._game_over_timer > 0:
            self._game_over_timer -= 1
        return

    if self._shoot_cooldown > 0:
        self._shoot_cooldown -= 1

    self.enemy_grid.move()

    # Roll once per frame, then pick a single enemy to fire.
    alive = self.enemy_grid.alive_enemies()
    if alive and random.random() < self.enemy_fire_chance:
        shooter = random.choice(alive)
        bullet_x = shooter.x + shooter.width // 2
        self.enemy_bullets.append(Bullet(bullet_x, shooter.y + shooter.height, direction=1))

    for bullet in self.player_bullets:
        bullet.move()
    for bullet in self.enemy_bullets:
        bullet.move()

    self.player_bullets = [b for b in self.player_bullets if not b.off_screen(self.height)]
    self.enemy_bullets = [b for b in self.enemy_bullets if not b.off_screen(self.height)]

```

```

self._resolve_player_bullet_hits()

for bullet in self.enemy_bullets:
    if bullet.rect().colliderect(self.player.rect()):
        self._trigger_game_over()
        break

if self.enemy_grid.reached_bottom(self.player.y):
    self._trigger_game_over()

def _render_game_over(self, screen):
    # Dim the frozen game behind the text
    overlay = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
    overlay.fill((0, 0, 0, 160))
    screen.blit(overlay, (0, 0))

    cx, cy = self.width // 2, self.height // 2

    title = self.title_font.render("GAME OVER", True, RED)
    screen.blit(title, title.get_rect(center=(cx, cy - 160)))

    score = self.font.render(f"Final Score: {self.score}", True, WHITE)
    screen.blit(score, score.get_rect(center=(cx, cy - 100)))

    # Menu appears once input is being accepted
    if not self._game_over_ready():
        return

    header = self.prompt_font.render("Play again? Choose a difficulty:", True, WHITE)
    screen.blit(header, header.get_rect(center=(cx, cy - 45)))

    for i, (label, hint) in enumerate(MENU_OPTIONS):
        selected = (i == self._menu_index)
        color = YELLOW if selected else WHITE
        marker = "> " if selected else " "
        y = cy + i * 45

        text = self.font.render(f"{marker}{label}", True, color)
        screen.blit(text, text.get_rect(midleft=(cx - 130, y)))

        hint_text = self.prompt_font.render(f"[{hint}]", True, GREY)
        screen.blit(hint_text, hint_text.get_rect(midright=(cx + 140, y)))

    help_text = self.prompt_font.render("Up/Down or W/S to move, Enter to select", True, GREY)
    screen.blit(help_text, help_text.get_rect(center=(cx, cy + 4 * 45 + 20)))

def render(self, screen):
    pygame.draw.rect(screen, GREEN, self.player.rect())

    for enemy in self.enemy_grid.alive_enemies():
        pygame.draw.rect(screen, WHITE, enemy.rect())

    for bullet in self.player_bullets:
        pygame.draw.rect(screen, WHITE, bullet.rect())
    for bullet in self.enemy_bullets:
        pygame.draw.rect(screen, RED, bullet.rect())

    score_text = self.font.render(f"Score: {self.score}", True, WHITE)
    screen.blit(score_text, (10, 10))

    if self.game_over:
        self._render_game_over(screen)

```

What to check for your "after" video (make sure your volume is up, since screen recorders need system audio enabled):

Pressing Space plays a short "pew". Holding Space during the cooldown doesn't re-trigger it.

Destroying an enemy plays a low explosion burst.

Getting hit, or the grid reaching the bottom, plays a descending four-note jingle once. It does not repeat while the game-over screen is showing.

Choosing a difficulty starts the new game silently.

Things to know:

Testing the fallback: To test the no-audio path, temporarily put raise RuntimeError as the first line inside the try: in SoundManager.__init__. The game should run silently with no errors. Remove the line afterwards.

Volume: Adjust it with MASTER_VOLUME at the top of sounds.py.

Folder structure: The README's folder tree doesn't list sounds.py, so the new file just goes next to game_engine.py in game/.
Startup time: Generating the sounds takes a few tens of milliseconds at launch, which is not noticeable.
That covers all four tasks. If you'd like, I can help you review the whole set of changes before you record the final videos.